

# Machine Learning Reviewer: Modules 1 and 2

---

CS0075 Machine Learning Algorithm. Built from the Module 1 and Module 2 slides plus the 17 verified formative questions. Read top to bottom; the blue boxes are memory aids I made (they are not from the slides), and the red boxes are the traps.

## How To Use This Reviewer

SA1 covers **Module 1 and Module 2 only up to Linear Regression (OLS)**. The midterm lecture exam on **Sat Oct 10** covers all lessons, so the sections are tagged:

| Tag           | Meaning                                  | Where                                                                                 |
|---------------|------------------------------------------|---------------------------------------------------------------------------------------|
| SA1 + Midterm | Covered by SA1 and by the midterm        | Parts 1, 2, and 3.1 to 3.3 (up to Linear Regression)                                  |
| Midterm only  | Not in SA1, but expect it on the midterm | Parts 3.4 onward (Ridge, Lasso, Logistic, Naive Bayes, trees, forests, boosting, SVM) |

The four parts follow the formative structure: **(1) core concepts, (2) evaluation and complexity, (3) the algorithms, (4) the traps**. The cram sheet is the last page.

## Part 1. Core Concepts and Definitions [SA1 + Midterm]

### What Machine Learning Is

- Machine learning is about **extracting knowledge from data**. It sits at the intersection of **statistics, artificial intelligence, and computer science**, and is also known as **predictive analytics** or **statistical learning**.
- Arthur Samuel, 1959**: a field of study concerned with giving computers the ability to **learn without being explicitly programmed**.
- ML is a discipline of AI that lets machines **automatically learn from data and past experiences**, identify patterns, and make **predictions** with **minimal human intervention**.

### The Formal Definition (E, T, P)

A program learns from **experience E** with respect to a class of **tasks T** and **performance measure P** if its performance at T, as measured by P, **improves with experience E**.

#### Memory Aid

Experience, Task, Performance. Mnemonic: **Every Try Pays** off (more experience, better score).

## Rule-Based Algorithm vs Machine Learning

|                | Traditional Rule-Based Algorithm           | Machine Learning            |
|----------------|--------------------------------------------|-----------------------------|
| What drives it | A <b>condition</b> (rules a person writes) | <b>Data</b> (or experience) |
| What goes in   | Rules + data                               | Data + answers              |
| What comes out | Answers                                    | The rules (the model)       |

### Memory Aid

Rule-based: **R**ules in, **A**nswers out. ML flips it: **D**ata and answers in, the **R**ules out.  
Fill-in-the-blank: "Rule-based is Condition. Machine learning is **Data**."

## Why Machine Learning

- Computers operate **autonomously without explicit programming**; fed new data they learn, grow, and adapt.
- Performance **adaptively improves as the number of available samples increases**.
- Machines work **24/7**, need no breaks, cost much less to maintain than a human in the long run, and suit tasks that are **routine, repetitive, or tedious**.
- Automation also **mitigates risks caused by fatigue or inattention**. Examples in the slides: self-driving cars and robotic arms on production lines.

## Types of Machine Learning and the Workflow

Three learning types: **Supervised, Unsupervised, Reinforcement**. The workflow has two phases:

| Phase                      | Steps                                                                                                                                                                          |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Phase 1: Learning</b>   | Training data, <b>preprocessing</b> (clean data, format data), <b>learning</b> (supervised, unsupervised, reinforcement), <b>testing</b> (measure performance, test algorithm) |
| <b>Phase 2: Prediction</b> | <b>New data</b> goes into the <b>trained model</b> , which outputs <b>predicted data</b>                                                                                       |

## Python Tools for ML

| Name                    | What The Slide Says                                                                                  |
|-------------------------|------------------------------------------------------------------------------------------------------|
| <b>Scikit-learn</b>     | The most prominent Python library for ML                                                             |
| <b>NumPy</b>            | A fundamental package for scientific computing                                                       |
| <b>SciPy</b>            | A collection of functions for scientific computing                                                   |
| <b>Matplotlib</b>       | The primary scientific plotting library                                                              |
| <b>Pandas</b>           | A library for data wrangling and analysis                                                            |
| <b>Anaconda</b>         | A Python distribution for large-scale data processing, predictive analysis, and scientific computing |
| <b>Jupyter Notebook</b> | An interactive environment for running code in the browser                                           |

### Watch Out

Formative Q5 ("most popular libraries, choose all") keyed **PANDAS, SCIPY, NUMPY**. Anaconda is a **distribution** and Jupyter is an **interactive environment**, so they are not libraries. SQL is a query language.

### Data Preprocessing

Real data has three quality problems: **incompleteness** (missing values or attributes), **noise** (incorrect records or exceptions), and **inconsistency** (inconsistent records). "**Without good data, there is no good model.**" Preprocessing (also called data preparation) is cleaning, altering, and reorganizing raw data before analysis.

| Category            | Techniques                                                                                                   |
|---------------------|--------------------------------------------------------------------------------------------------------------|
| Data Cleansing      | Identify and sort out <b>missing data</b> , reduce <b>noisy data</b> , identify and remove <b>duplicates</b> |
| Feature Engineering | <b>Feature scaling or normalization, data reduction, discretization, feature encoding</b>                    |

### Exploratory Data Analysis (EDA)

EDA understands the main characteristics of the data, spots patterns and **anomalies**, tests a hypothesis, or checks assumptions, often with visualization.

| Activity             | What It Does                                        |
|----------------------|-----------------------------------------------------|
| Visualization        | Plots and charts of distributions and relationships |
| Summary Statistics   | Mean, median, variance, standard deviation          |
| Outlier Detection    | Identifying <b>unusual data points</b>              |
| Correlation Analysis | Examining relationships between variables           |
| Hypothesis Testing   | Testing initial assumptions about the data          |

### Memory Aid

An **outlier** is the point; **outlier detection** is the process. If the blank says "identifies unusual data points", answer the process.

### Discrete vs Continuous Data

| Type       | Meaning                                                    | Example                       |
|------------|------------------------------------------------------------|-------------------------------|
| Discrete   | Countable in whole units; you cannot have a fraction of it | Number of students in a class |
| Continuous | Lies on an unbroken scale                                  | Height, weight, temperature   |

### Memory Aid

**Discrete = Digits you can count. Continuous = Can slide** anywhere on the scale.

## Model Deployment and Data Ethics (Module 1, Subtopic 2)

**Deployment** means taking a trained model and making it available in real applications. The stages are **Training, Validation, Deployment, Monitoring** (models can **drift** as real-world data evolves). Best practices: (1) **choose the right infrastructure**, (2) **effective versioning and tracking**, (3) **robust testing and validation**, (4) **implement monitoring and alerting**.

| Data Ethics Principle | Meaning                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Ownership</b>      | An individual owns their personal information; collecting it without consent is unethical                                         |
| <b>Transparency</b>   | Data subjects have a right to know how it is collected, stored, and used                                                          |
| <b>Privacy</b>        | Protect personally identifiable information (PII) such as full name, birthdate, phone number, bank account number, street address |
| <b>Intention</b>      | Ask why you need the data and what you will gain; malicious intent is unethical                                                   |
| <b>Outcomes</b>       | Even good intentions can cause harm; unintended harm to a group is called <b>disparate impact</b>                                 |

### Memory Aid

**Own it, Tell them, Protect it, Intend well, check Outcomes: Our Team Protects Individuals' Ownership.**

## Part 2. Evaluation, Generalization and Complexity [SA1 + Midterm]

### Supervised Learning Basics

- **Labeled training data** has both the **input** and the **correct output** (a label). Each example is a pair: an input object (feature vector) and a desired output value.
- The primary objective is to **map the input variable to the output variable**.
- Two phases: **training and validation**, followed by **prediction**.

|                           | Classification                                                                               | Regression                                                                            |
|---------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Target</b>             | <b>Categorical</b> variable                                                                  | <b>Continuous</b> variable                                                            |
| <b>Predicts</b>           | One of the possible <b>class labels</b>                                                      | <b>Consecutive numbers (real numbers)</b> , per the slide                             |
| <b>Examples</b>           | Spam or not spam, COVID positive or negative on a chest X-ray, handwritten digit recognition | Stock price forecast, weather prediction, annual income forecast, population forecast |
| <b>Subtypes</b>           | <b>Binary</b> (two classes) and <b>multiple</b> (three or more)                              | Simple linear, multivariate                                                           |
| <b>Algorithms (slide)</b> | Random Forest, Decision Tree, Logistic Regression, Support Vector                            | Simple Linear Regression, Multivariate Regression, Decision                           |

|  |         |                        |
|--|---------|------------------------|
|  | Machine | Tree, Lasso Regression |
|--|---------|------------------------|

### Watch Out

**Formative Q17 conflict.** The slide says regression "**predicts consecutive numbers (real numbers)**", so the slide-literal answer to "Regression predicts consecutive numbers" is **True**. The Gemini key marked it **False** (regression predicts continuous values, not a counting sequence like 1, 2, 3). Check which one your own formative feedback accepted before the exam. Rule of thumb: on a quiz copied from these slides, follow the slide wording.

## Generalization, Overfitting, Underfitting

| Term                  | What The Slides Say                                                                                                                                                            | Train     | Test |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|------|
| <b>Generalization</b> | The model makes accurate predictions on <b>new, unseen data</b> with the same characteristics as the training set. We want to generalize <b>as accurately as possible</b> .    | Good      | Good |
| <b>Overfitting</b>    | The model fits the <b>particularities of the training set</b> (its noise and outliers) too closely; it has <b>memorized</b> the training data rather than learned the pattern. | Very good | Poor |
| <b>Underfitting</b>   | The model is <b>too simple</b> to capture the variability in the data and does badly <b>even on the training set</b> .                                                         | Poor      | Poor |

### Memory Aid

**Under = Underbuilt** (bad everywhere). **Over = Overthinks** (aces training, flops on new data).

## Model Complexity

- The more complex the model, the better it predicts the **training** data. Too complex, and it focuses on each individual point and **does not generalize**.
- There is a **sweet spot** in between that gives the **best generalization**. That is the model we want.
- The larger the **variety** of data points, the more complex a model you can use without overfitting. **Duplicating the same points or collecting very similar data does not help**.
- "Never underestimate the power of **more data**."

| Complexity                     | Fit Problem         | Error Type (Slide Definition)                                                           |
|--------------------------------|---------------------|-----------------------------------------------------------------------------------------|
| <b>Too low</b> (too simple)    | <b>Underfitting</b> | <b>Bias</b> = error from wrong or too-simple assumptions in the algorithm               |
| <b>Too high</b> (too flexible) | <b>Overfitting</b>  | <b>Variance</b> = error from sensitivity to noise and fluctuations in the training data |

**Watch Out**

"Low complexity is overfitting" is **False**. Low complexity is **underfitting**.

The boat example: a rule that is 100 percent accurate on 12 rows is not trustworthy. If 10,000 more rows obeyed the same rule, you would believe it. More data supports a more complex model.

**R-Squared (R2, Coefficient of Determination)**

| Situation                                                                                          | R2              |
|----------------------------------------------------------------------------------------------------|-----------------|
| <b>Perfect prediction</b> (prediction equals target)                                               | <b>1</b>        |
| <b>Constant model</b> that always predicts the mean of the training targets ( $y_{\text{train}}$ ) | <b>0</b>        |
| <b>Worse than predicting the average</b>                                                           | <b>Negative</b> |

**Memory Aid**

R2 is a **report card against guessing the average**: 1 is perfect, 0 is no better than the average, **below 0 is worse than the average**.

**Watch Out**

The slide also says R2 "yields a score between 0 and 1", but the next slide says it **can be negative**. "Predicting worse than the average can give a negative number" is **True** (formative Q11). For regressors, scikit-learn's **score()** returns **R2**; for classifiers it returns **accuracy**.

**Part 3. The Algorithms****3.1 k-Nearest Neighbors (k-NN) [SA1 + Midterm]**

- Arguably the **simplest** ML algorithm. **Building the model is only storing the training dataset**.
- To predict, it finds the **closest training points** (the nearest neighbors).
- Classification**: with  $k = 1$ , the prediction is the label of the single nearest point. With  $k > 1$  it uses **voting**: the **majority class** among the  $k$  neighbors wins. Works for any number of classes.
- Regression**: with  $k = 1$ , the prediction is the target of the nearest neighbor. With  $k > 1$  the prediction is the **average (mean)** of the neighbors.
- In scikit-learn: **KNeighborsClassifier** and **KNeighborsRegressor**; train with **fit**, predict with **predict**, evaluate with **score**.

|                          | Small k (for example 1)                                        | Large k                                                  |
|--------------------------|----------------------------------------------------------------|----------------------------------------------------------|
| <b>Decision boundary</b> | Jagged; predictions go through every point ( <b>unsteady</b> ) | <b>Smoother</b> ; does not fit the training data as well |
| <b>Model complexity</b>  | <b>More complex</b>                                            | <b>Less complex</b>                                      |

|             |                                    |                                 |
|-------------|------------------------------------|---------------------------------|
| <b>Risk</b> | <b>Overfitting</b> (high variance) | <b>Underfitting</b> (high bias) |
|-------------|------------------------------------|---------------------------------|

#### Memory Aid

**Small k = Small crowd = Super complex = Overfit. Big k = Big crowd = Blurry average = Underfit.** On the accuracy plot, **the best performance is somewhere in the middle.**

#### Watch Out

"A small k makes the model less complex" is **False** (formative Q9). "In k-NN, low model complexity is overfitting" is **False** (Q10): low complexity is underfitting.

| Important Parameters                                                                                                                          | Strengths                                                                                            | Weaknesses                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Number of neighbors</b> and the <b>distance measure</b> (Euclidean by default). Three or five neighbors usually works well, but adjust it. | Easy to understand; works well <b>without special adjustments</b> ; suitable as a <b>first model</b> | <b>Slow</b> prediction when features or samples are many, so preprocessing matters; does <b>not work well with sparse datasets</b> |

### 3.2 Linear Models [SA1 + Midterm]

- Linear models make a prediction using a **linear function of the input features**: a **best-fit line**.
- They assume a **linear relationship** between the outcome and each predictor. Examples: boiling point vs altitude, advertising spend vs revenue, fertilizer vs crop yield.
- Types: **Linear Regression** (regression, numeric output) and **Logistic Regression** (classification).

#### The Prediction Formula

| Symbol              | Meaning                                                             |
|---------------------|---------------------------------------------------------------------|
| <b>x[0] to x[p]</b> | The features ( <b>p + 1</b> of them)                                |
| <b>w[0] to w[p]</b> | The <b>weights</b> or <b>coefficients</b> (the slopes)              |
| <b>b</b>            | The <b>intercept</b> , also called the <b>offset</b> or <b>bias</b> |
| <b>y-hat</b>        | The model's prediction                                              |

#### Memory Aid

**w = Weight = slope. b = Bias = intercept = offset.** All three of intercept, offset, and bias point at **b**.

In scikit-learn the weights are stored in **coef\_** and the intercept in **intercept\_**. "The offset parameter is also called the intercept" is **True** (formative Q12).

### 3.3 Linear Regression (Ordinary Least Squares) [SA1 + Midterm]

- Linear regression = ordinary least squares (OLS)**: the simplest and most classic linear method for regression.

- It finds **w** and **b** that **minimize the mean squared error** between predictions and the true targets on the **training set**.
- The **mean squared error** is the sum of squared differences between the predictions and the true values (as the slide words it).

| Example                                                            | Train R2 vs Test R2                                                     | Diagnosis                                                                                                       |
|--------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Wave dataset</b> (one feature)                                  | About <b>0.66</b> , and the train and test scores are <b>very close</b> | Likely <b>underfitting</b> , not overfitting                                                                    |
| <b>Extended Boston Housing</b> (506 samples, <b>105 features</b> ) | <b>Very accurate on train</b> , much <b>worse on test</b>               | <b>Overfitting</b> . Too many features inflate the weights. Try a model that <b>controls complexity</b> : Ridge |

#### Memory Aid

Close scores, mediocre = underfit. Big gap, high train = overfit.

SA1 stops here. Everything below is **midterm only**.

### 3.4 Ridge Regression [Midterm only]

- A linear model for regression with the **same prediction formula as OLS**.
- The coefficients are also chosen to be **as small as possible** (close to zero, **but not zero**), so each feature has as little effect as it can while still predicting well.
- That constraint is **regularization: explicitly restricting a model to avoid overfitting**. Ridge uses **L2 regularization** (squared L2 norm).
- **Alpha** controls the strength. **Larger alpha, larger penalty, smaller w**. Default **alpha = 1.0**.

|                    | Ridge vs LinearRegression (Boston)                                                                                         |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| Training score     | Ridge is <b>lower</b>                                                                                                      |
| Test score         | Ridge is <b>higher</b> (less overfitting; Ridge is a more restricted model)                                                |
| More training data | The gap closes; with enough data <b>regularization matters less</b> . OLS at tiny sizes could even score <b>R2 below 0</b> |

#### Memory Aid

Alpha up, **All** weights shrink toward zero. A bigger alpha is a **bigger fine**.

#### Watch Out

"In Ridge, a larger alpha makes the penalty lesser" is **False** (formative Q13). Ridge weights **approach zero but never reach it**.

### 3.5 Lasso [Midterm only]

- An alternative to Ridge. It also keeps coefficients near zero, but with **L1 regularization**.
- Consequence: **some coefficients become exactly zero**, so some features are **ignored entirely**.
- Larger alpha again means a larger penalty. **Alpha = 1.0 was too much regularization: underfitting**, using only **4 of the 105 features**. **Lowering alpha** to reduce underfitting used **33 of 105 features** and did slightly better than Ridge. Very low alpha behaves **like plain linear regression**.

|                | Ridge                                    | Lasso                                                                                                        |
|----------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Penalty        | <b>L2</b>                                | <b>L1</b>                                                                                                    |
| Weights        | Approach zero, <b>never exactly zero</b> | <b>Can be exactly zero</b>                                                                                   |
| Prefer it when | <b>Generally preferred</b>               | Only <b>some of many features matter</b> , or you want a model that is <b>easy to analyze and understand</b> |

#### Memory Aid

**L1 = Leaves out features** (zeros). **L2 = Loves them all a little** (shrinks, keeps).

### 3.6 Logistic Regression and LinearSVC [Midterm only]

- Linear models for **classification**. For binary classification, the prediction is positive (+1) if the formula is **greater than 0** and negative (-1) if **less than 0**.
- The decision boundary is a **line** (1 feature), a **plane** (2 features), or a **hyperplane** (3 or more).
- **Logistic regression** is a classification algorithm despite its name. It gives a **probability between 0 and 1** using the **logistic (sigmoid) function**, and a **threshold** decides the class.
- The two common linear classifiers: **LogisticRegression** and **linear SVM (LinearSVC)**.
- Regularization parameter **C**: **higher C = less regularization** (fits the training set as well as it can); **low C** pushes **w toward zero**. Default **C = 1** with **L2**; **L1** gives a more **interpretable** model using fewer features.

#### Watch Out

C works **opposite** to alpha: **high C = weak regularization**, **high alpha = strong regularization**.

#### Memory Aid

**C = Control** is loose when high. **Alpha = Accuse** harder when high.

### 3.7 Naive Bayes [Midterm only]

- A family of classifiers quite similar to linear models; **training is faster** than a linear classifier, with slightly lower generalization.
- They are efficient because they learn parameters by looking at **each feature individually** and collecting **simple per-class statistics** from each feature (formative Q15: **True**).
- **Alpha** controls model complexity by smoothing statistics. **Larger alpha decreases complexity** but does not change performance much.
- Works well with **sparse, high-dimensional** data and is **not parameter sensitive**. Training and prediction are fast and easy to understand.

| Classifier    | Data                          |
|---------------|-------------------------------|
| GaussianNB    | Continuous data               |
| BernoulliNB   | Binary data, text data        |
| MultinomialNB | Integer count data, text data |

#### Memory Aid

Gaussian = **G**rades on a scale (continuous). Bernoulli = **B**inary. Multinomial = **M**any counts.

### 3.8 Decision Trees [Midterm only]

- Used for **classification and regression**. They learn a **hierarchy of if/else questions** leading to a decision.
- The questions are called **tests** (not the test set). The top node is the **root** (the whole dataset); the algorithm picks the **most informative** test about the target. Parts: **root node, node, edge, terminal (leaf) node**.
- A prediction finds the region the point falls in and returns the **majority target** (or the single target in a pure leaf).
- scikit-learn: **DecisionTreeClassifier** and **DecisionTreeRegressor**. Visualize with **export\_graphviz** (a **.dot** file). **Feature importances** show what the tree used.
- Tree regression is **not affected by feature scale**, but it is **impossible to extrapolate**: it cannot predict outside the range of the training data.

### 3.9 Ensembles: Random Forest and Gradient Boosting [Midterm only]

Ensembles combine multiple models into a more powerful one. Both below use **decision trees** as building blocks.

|                       | Random Forest                                                                                                                      | Gradient Boosting                                                                                            |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Idea</b>           | <b>Reduces overfitting by averaging</b> many trees; <b>injects randomness</b>                                                      | Each new tree <b>compensates for the error of the previous tree</b>                                          |
| <b>Trees</b>          | Built <b>independently</b> ; each from a <b>bootstrap sample</b> (draw with replacement; about <b>one third</b> of points missing) | <b>Shallow</b> trees, depth <b>5 or less</b> ; <b>pre-pruning</b> is applied, not randomness                 |
| <b>Prediction</b>     | Regression: <b>average of values</b> .<br>Classification: <b>average of predicted probabilities</b>                                | Regression: <b>least squares</b> loss.<br>Classification: <b>logistic</b> loss. Uses <b>gradient descent</b> |
| <b>Key parameters</b> | <b>n_estimators</b> , <b>max_features</b>                                                                                          | <b>n_estimators</b> , <b>learning_rate</b> (default <b>0.1</b> ), <b>max_depth</b>                           |
| <b>Strengths</b>      | Widely used; excellent performance; little tuning; <b>no scaling needed</b>                                                        | Slightly <b>better than random forest</b> ; no scaling needed                                                |
| <b>Weaknesses</b>     | Hard to analyze; poor on <b>large sparse</b> data; <b>more memory and slower</b> than linear models                                | <b>Sensitive to parameters</b> ; longer training; poor on sparse high-dimensional data                       |

#### Memory Aid

**Forest = many independent trees VOTE (average). Boosting = trees take TURNS fixing the last one's mistakes.**

On the Breast Cancer data: random forest gave **97 percent** with no tuning; gradient boosting at defaults hit **100 percent on training**, a sign of **overfitting**, fixed by **limiting depth** or **lowering the learning rate**.

### 3.10 Kernelized Support Vector Machines [Midterm only]

- An extension of linear SVM for **nonlinear** problems that a hyperplane cannot solve. **SVC** for classification, **SVR** for regression.
- **Support vectors** are the points that define the decision boundary. The **kernel trick** gives complex boundaries even with few features.
- **gamma** controls the width of the **Gaussian (RBF) kernel**: **small gamma = large radius = many points count as close = simpler**.
- **C** limits each point's influence: **small C = very restricted model**.
- SVMs are **sensitive to feature scale**. Rescale to **0 to 1** with **MinMaxScaler**. On the Breast Cancer data this fixed the overfitting, then **increasing C** reached **97.2 percent**.
- Weakness: **slow with many samples (more than about 100,000)**; sensitive to preprocessing and parameters. Parameters: **C**, **gamma** (RBF), **coef** (sigmoid), **degree** (polynomial).

## Part 4. High-Yield Traps

These are the twists the formatives used. Most are a true idea with **one word flipped**.

| Trap                      | Wrong Idea                            | Right Idea                                                                                                                    |
|---------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Discrete vs continuous    | Number of students is continuous      | <b>Discrete</b> : countable whole units                                                                                       |
| Consecutive vs continuous | Slide wording vs Gemini key           | Slide: regression predicts <b>consecutive (real) numbers</b> . Key said <b>False</b> . <b>Check your formative feedback</b> . |
| Small k                   | Small k makes the model less complex  | Small k = <b>more</b> complex = overfitting. <b>False</b> .                                                                   |
| Low complexity            | Low complexity is overfitting         | Low complexity is <b>underfitting</b> . <b>False</b> .                                                                        |
| Rule-based vs ML          | ML runs on conditions                 | Rule-based = <b>condition</b> ; ML = <b>data</b>                                                                              |
| Ridge alpha               | Larger alpha makes the penalty lesser | Larger alpha = <b>bigger</b> penalty. <b>False</b> .                                                                          |
| Alpha vs C                | High C means strong regularization    | <b>High C = weak</b> regularization; high alpha = strong                                                                      |
| Ridge vs Lasso zeros      | Ridge zeroes out features             | <b>Lasso</b> (L1) hits exact zero; <b>Ridge</b> (L2) only approaches it                                                       |
| Naive Bayes types         | GaussianNB for counts                 | Counts = <b>Multinomial</b> ; continuous = <b>Gaussian</b> ; binary = <b>Bernoulli</b>                                        |
| R2 sign                   | R2 can never be negative              | Worse than the mean gives <b>negative</b> R2. <b>True</b> .                                                                   |
| Offset                    | Offset is not the intercept           | Offset = intercept = bias ( <b>b</b> ). <b>True</b> .                                                                         |
| Logistic Regression       | A regression algorithm                | A <b>classification</b> algorithm despite its name                                                                            |
| Libraries                 | Anaconda or Jupyter is a library      | Libraries: <b>pandas, SciPy, NumPy</b> (also scikit-learn, Matplotlib).<br>Anaconda = distribution; Jupyter =                 |

|              |                                     |                                                             |
|--------------|-------------------------------------|-------------------------------------------------------------|
|              |                                     | environment.                                                |
| Outlier word | Answering the process for the point | Point = <b>outlier</b> ; process = <b>outlier detection</b> |
| score()      | score() is always accuracy          | <b>Classifier = accuracy; regressor = R2</b>                |

## Cram Sheet

Everything on one page. **SA1 items first**, midterm-only items after the line.

- **ML** = extracting knowledge from data; learns from **data / experience** to predict with minimal human intervention. **Samuel, 1959**.
- **E, T, P**: performance at task T, measured by P, improves with experience E.
- **Rule-based = condition; ML = data**.
- Workflow: **Learning** (preprocess, learn, test) then **Prediction** (new data into trained model).
- **Discrete** = countable. **Continuous** = unbroken scale. **Outlier** = far from average; finding it = **outlier detection** (EDA).
- Libraries: **pandas, SciPy, NumPy**, scikit-learn, Matplotlib. Anaconda = distribution, Jupyter = environment.
- Preprocessing: **cleansing** (missing, noise, duplicates) and **feature engineering** (scaling, reduction, discretization, encoding).
- **Classification** = categorical target. **Regression** = continuous target (slide: consecutive real numbers).
- **Generalization** = good on unseen data. **Underfitting** = too simple, **high bias**, poor everywhere. **Overfitting** = memorized training, **high variance**.
- **Small k** = complex = overfit. **Large k** = simple = underfit. k-NN: store data, vote (class) or average (regression), Euclidean by default.
- **R2**: 1 perfect, 0 = predicts the mean, **negative = worse than the mean**.
- **y-hat = w\*x + b**; **b** = intercept = offset = bias. **OLS** minimizes mean squared error on the training set.
- OLS: small gap and mediocre score = underfit; huge train-test gap (105 features) = overfit.
- ---- SA1 ends at linear regression. Midterm-only below. ----
- **Ridge** = L2, weights shrink but never zero; **larger alpha = larger penalty**. **Lasso** = L1, some weights **exactly zero**.
- **Logistic Regression** = classification (sigmoid, threshold). **C high = less regularization**.
- **Naive Bayes**: each feature alone, per-class stats. **Gaussian** continuous, **Multinomial** counts, **Bernoulli** binary.
- **Decision tree** = if/else tests, cannot extrapolate. **Random forest** = average of independent bootstrapped trees. **Gradient boosting** = shallow trees fixing the previous error, learning rate 0.1.
- **SVM** = kernel trick for nonlinear data; **scale to 0-1**; small gamma = simpler; small C = restricted.